```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

chibichip_guidelines_path = kagglehub.dataset_download('chibichip/guidelines')

print('Data source import complete.')
```

```
# 1. Install high-performance inference and quantization libraries
!pip install -q -U transformers accelerate bitsandbytes

# 2. Install PDF processing and Graph management tools
!pip install -q pypdf networkx pyvis

# 3. Import the necessary modules for our pipeline
import torch
import json
import gc
import networkx as nx
from pypdf import PdfReader
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
from pyvis.network import Network
from IPython.display import HTML

# Initialize a clean MultiDiGraph for our data
G = nx.MultiDiGraph()

print("✅ Step 1 Complete: Environment is clean and libraries are ready.")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.7/10.7 MB 110.6 MB/s eta 0:00:0000:010:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 383.7/383.7 kB 23.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 28.1 MB/s eta 0:00:00:00:0100:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 756.0/756.0 kB 18.0 MB/s eta 0:00:0000:01
✅ Step 1 Complete: Environment is clean and libraries are ready.
```

```
from transformers import BitsAndBytesConfig

# 1. Define the 4-bit configuration
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

# 2. Initialize the Tokenizer
model_id = "Qwen/Qwen2.5-7B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id)

# 3. Load the Model using the bnb_config
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    quantization_config=bnb_config, # This is the fix
    torch_dtype=torch.float16
)

# 4. Create the pipeline
local_llm = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=512,
    do_sample=False,
    temperature=0.0
)

print("✅ Step 2 Complete: Qwen 2.5 7B is successfully loaded in 4-bit!")
```

```
Loading weights:   0%|          | 0/339 [00:00<?, ?it/s]
generation_config.json:   0%|          | 0.00/243 [00:00<?, ?B/s]
The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS_VERBOS
Passing `generation_config` together with generation-related arguments=({'temperature', 'do_sample', 'max_new_tokens'}) is c
✅ Step 2 Complete: Qwen 2.5 7B is successfully loaded in 4-bit!
```

```
import re

# 1. Function to extract triples from text
def extract_graph_triples(text_chunk):
    # Prompt tailored for the Digital Lending Guidelines
    prompt = f"""<|im_start|>system
You are a regulatory compliance auditor. Extract a knowledge graph from the provided RBI Digital Lending Guidelines.
Focus on entities like Regulated Entities (REs), Lending Service Providers (LSPs), Digital Lending Apps (DLAs), and Borrowe
Capture rules regarding: Disbursal, Fees, Disclosures (KFS/APR), and Data Privacy.

Return ONLY a valid JSON object:
{{
  "edges": [
    {{"source": "Entity A", "target": "Entity B", "relationship": "Action/Rule"}}
  ]
}}
<|im_end|>
<|im_start|>user
Text: {text_chunk}
<|im_end|>
<|im_start|>assistant
"""
    # Generate the response
    outputs = local_llm(prompt, max_new_tokens=512)
    raw_content = outputs[0]['generated_text'].split("assistant\n")[-1]

    # 2. Robust JSON Parsing
    try:
        # Use regex to find the JSON block if the model adds extra text
        json_match = re.search(r'\{.*\}', raw_content, re.DOTALL)
        if json_match:
            data = json.loads(json_match.group())
            return data.get("edges", [])
        return []
    except Exception as e:
        print(f"Parsing error on a chunk: {e}")
        return []

print("✅ Step 3 Complete: Extraction logic defined and JSON parser ready.")
```

```
✅ Step 3 Complete: Extraction logic defined and JSON parser ready.
```

```
# 1. Load the Digital Lending PDF
pdf_path = "/kaggle/input/datasets/chibichip/guidelines/GUIDELINES ON DIGITAL LENDING.pdf"
reader = PdfReader(pdf_path)
total_pages = len(reader.pages)

print(f"📖 Starting extraction for {total_pages} pages...")

# 2. The Extraction Loop
for i in range(total_pages):
    # Extract text from the current page
    page_text = reader.pages[i].extract_text()

    # Run the extraction logic from Step 3
    edges = extract_graph_triples(page_text)

    # Add extracted edges to our global graph G
    for edge in edges:
        G.add_edge(
            edge['source'],
            edge['target'],
            title=edge['relationship']
        )

    print(f"✅ Processed Page {i+1}/{total_pages} | Total Edges: {len(G.edges())}")

    # 3. Memory Management (Critical for P100)
    torch.cuda.empty_cache()
    gc.collect()

print("\n🚀 Full PDF Extraction Complete!")
print(f"Final Graph Stats: {len(G.nodes())} Nodes, {len(G.edges())} Relationships.")
```

```
Passing `generation_config` together with generation-related arguments=({'max_new_tokens'}) is deprecated and will be remove
📖 Starting extraction for 12 pages...
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 1/12 | Total Edges: 9
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 2/12 | Total Edges: 13
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 3/12 | Total Edges: 21
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 4/12 | Total Edges: 25
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 5/12 | Total Edges: 31
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 6/12 | Total Edges: 41
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 7/12 | Total Edges: 52
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 8/12 | Total Edges: 63
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 9/12 | Total Edges: 70
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 10/12 | Total Edges: 72
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 11/12 | Total Edges: 87
Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
✅ Processed Page 12/12 | Total Edges: 91

🚀 Full PDF Extraction Complete!
Final Graph Stats: 29 Nodes, 91 Relationships.
```

```
apr_logic = [
    {"source": "APR", "target": "Cost of Funds", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Credit Cost", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Operating Cost", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Processing Fee", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Verification Charges", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Maintenance Charges", "relationship": "MUST_INCLUDE"},
    {"source": "APR", "target": "Penal Charges", "relationship": "MUST_EXCLUDE"},
    {"source": "APR", "target": "Late Payment Charges", "relationship": "MUST_EXCLUDE"}
]

for edge in apr_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print(f"✅ Logic Updated: {len(apr_logic)} components added to the APR node.")
```

```
✅ Logic Updated: 8 components added to the APR node.
```

```
# Create the connections between the APR cluster and the main entities
bridge_logic = [
    # REs are responsible for the APR disclosure
    {"source": "Regulated Entity (RE)", "target": "APR", "relationship": "MUST_DISCLOSE_UPFRONT"},

    # APR must be part of the Key Fact Statement (KFS)
    {"source": "Key Fact Statement (KFS)", "target": "APR", "relationship": "MUST_CONTAIN"},

    # Borrowers are the ones who must receive this info
    {"source": "APR", "target": "Borrower", "relationship": "EFFECTIVE_ANNUALIZED_RATE_FOR"}
]

for edge in bridge_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print("✅ Clusters bridged! APR is now logically connected to REs and Borrowers.")
```

```
✅ Clusters bridged! APR is now logically connected to REs and Borrowers.
```

```
# 1. Define the missing operational and privacy logic from the PDF
operational_logic = [
    # Cooling-off / Exit rules (Page 7)
    {"source": "Cooling-off Period", "target": "Loan (Tenor >= 7 days)", "relationship": "MINIMUM_3_DAYS"},
    {"source": "Cooling-off Period", "target": "Loan (Tenor < 7 days)", "relationship": "MINIMUM_1_DAY"},
    {"source": "Borrower", "target": "Cooling-off Period", "relationship": "CAN_EXIT_BY_PAYING_PRINCIPAL_AND_APR"},

    # Data Privacy Prohibitions (Page 7-8)
    {"source": "Digital Lending App (DLA)", "target": "Contact List", "relationship": "MUST_DESIST_FROM_ACCESSING"},
    {"source": "Digital Lending App (DLA)", "target": "Media and Files", "relationship": "MUST_DESIST_FROM_ACCESSING"},
    {"source": "Digital Lending App (DLA)", "target": "Call Logs", "relationship": "MUST_DESIST_FROM_ACCESSING"},
    {"source": "Digital Lending App (DLA)", "target": "Biometric Data", "relationship": "NO_STORAGE_ALLOWED"},
    {"source": "Digital Lending App (DLA)", "target": "One-time Access (Camera/Microphone)", "relationship": "ONLY_FOR_KYC_
```

```
    # Disbursement Hard-Stops (Page 4)
    {"source": "Regulated Entity (RE)", "target": "Borrower Bank Account", "relationship": "MUST_DISBURSE_DIRECTLY_TO"},
    {"source": "Lending Service Provider (LSP)", "target": "Disbursement Funds", "relationship": "MUST_NOT_RECEIVE_IN_POOL_
]

# 2. Add these to the graph
for edge in operational_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print(f"✅ Added {len(operational_logic)} operational rules. Now re-run your visualization cell!")
```

```
✅ Added 10 operational rules. Now re-run your visualization cell!
```

```
# Adding Enforcement and Consequences logic
enforcement_logic = [
    # The Golden Rule of KFS
    {"source": "Key Fact Statement (KFS)", "target": "Undisclosed Charges", "relationship": "CANNOT_BE_RECOVERED_FROM_BORR(

    # Regulatory Consequences
    {"source": "Regulated Entity (RE)", "target": "Missing KFS/APR", "relationship": "SUBJECT_TO_PENAL_ACTION_BY_RBI"},
    {"source": "Missing KFS/APR", "target": "Complaints", "relationship": "ESCALATED_TO_CMS_PORTAL"},

    # Standardized format rule
    {"source": "Key Fact Statement (KFS)", "target": "Standardized RBI Format", "relationship": "MUST_ADHERE_TO"}
]

for edge in enforcement_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print("🛡️ Enforcement logic added! The graph now knows the consequences of non-compliance.")
```

```
🛡️ Enforcement logic added! The graph now knows the consequences of non-compliance.
```

```
# Adding the Blanket Prohibition for undisclosed charges
blanket_rule_logic = [
    # Any fee not in KFS is illegal to collect
    {"source": "Key Fact Statement (KFS)", "target": "All-Inclusive Fees", "relationship": "MUST_DISCLOSE_EVERY_SINGLE_CHAI
    {"source": "Any Fee Not in KFS", "target": "Borrower", "relationship": "CANNOT_BE_CHARGED_TO"},
    {"source": "Convenience Fee", "target": "Any Fee Not in KFS", "relationship": "IS_A_TYPE_OF"},

    # Connection to the RE's responsibility
    {"source": "Regulated Entity (RE)", "target": "Key Fact Statement (KFS)", "relationship": "MUST_ENSURE_COMPLETENESS_OF'
]

for edge in blanket_rule_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print("🚫 'Blanket Prohibition' added. The graph now blocks any undisclosed fee by default.")
```

```
🚫 'Blanket Prohibition' added. The graph now blocks any undisclosed fee by default.
```

```
from pyvis.network import Network
import IPython

# 1. Initialize Pyvis Network
# We use 'cdn_resources=remote' to ensure the Javascript library loads inside Kaggle
net = Network(notebook=True, height="600px", width="100%", bgcolor="#1a1a1a", font_color="white", cdn_resources='remote')

# 2. Load the NetworkX graph into Pyvis
net.from_nx(G)

# 3. Apply some "Physics" so the graph spreads out nicely
net.toggle_physics(True)

# 4. Save and Display
net.save_graph("digital_lending_graph.html")
IPython.display.HTML(filename="digital_lending_graph.html")
```

```
def query_graph_v2(user_query):
    # 1. Prepare the context by listing the relationships found in the PDF
    relationships = list(set([f"{u} --({data['title']})--> {v}" for u, v, data in G.edges(data=True)]))
    graph_context = "\n".join(relationships)

    # 2. Construct the prompt using the Digital Lending context
    prompt = f"""<|im_start|>system
You are a Regulatory Expert. Answer the question using ONLY the Knowledge Graph relationships provided below.
If the information is not in the graph, say you don't have enough data.

Knowledge Graph:
{graph_context}
<|im_end|>
<|im_start|>user
Question: {user_query}
<|im_end|>
<|im_start|>assistant
"""
    # 3. Generate the answer
    with torch.no_grad():
        outputs = local_llm(prompt, max_new_tokens=300, do_sample=False)

    return outputs[0]['generated_text'].split("assistant\n")[-1]

print("🔍 Query Engine defined! You can now run the test query.")
```

```
🔍 Query Engine defined! You can now run the test query.
```

```
test_query = "Who is responsible for the grievance redressal if a Lending Service Provider (LSP) makes a mistake?"
print(f"Question: {test_query}")
print("Answer:", query_graph_v2(test_query))
```

```
Passing `generation_config` together with generation-related arguments=({'do_sample', 'max_new_tokens'}) is deprecated and w
Both `max_new_tokens` (=300) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
Question: Who is responsible for the grievance redressal if a Lending Service Provider (LSP) makes a mistake?
Answer: The responsibility for grievance redressal if a Lending Service Provider (LSP) makes a mistake lies with the Regulat

1. **Regulated Entity (RE) --(Ensure resolution of complaints within 30 days)--> Borrowers**
2. **Borrowers --(can lodge complaint if not resolved within 30 days)--> Complaint Management System (CMS) portal**
3. **Regulated Entity (RE) --(Ensure compliance)--> Lending Service Providers (LSPs)**
4. **Regulated Entity (RE) --(Conduct enhanced due diligence before partnering)--> Lending Service Provider (LSP)**
5. **Regulated Entity (RE) --(Ensure LSPs comply with extant instructions)--> LSP**

From these relationships, it is clear that the Regulated Entity (RE) is responsible for ensuring that LSPs comply with guide
```

```
# 1. Remove the vague tenor nodes
nodes_to_remove = ["Loan (Tenor >= 7 days)", "Loan (Tenor < 7 days)"]
for node in nodes_to_remove:
```

```
    if node in G:
        G.remove_node(node)

# 2. Add highly specific, mathematical rules
precision_logic = [
    {"source": "Cooling-off Period", "target": "Tenor 7 days or more", "relationship": "DURATION_MUST_BE_MINIMUM_3_DAYS"},
    {"source": "Cooling-off Period", "target": "Tenor less than 7 days", "relationship": "DURATION_MUST_BE_MINIMUM_1_DAY"},
    {"source": "10-day Loan", "target": "Tenor 7 days or more", "relationship": "IS_CATEGORIZED_AS"},
    {"source": "6-day Loan", "target": "Tenor less than 7 days", "relationship": "IS_CATEGORIZED_AS"}
]

for edge in precision_logic:
    G.add_edge(edge['source'], edge['target'], title=edge['relationship'])

print("🎯 Accuracy Refined: Added categorical bridges to prevent math errors.")
```

```
🎯 Accuracy Refined: Added categorical bridges to prevent math errors.
```

```
import ipywidgets as widgets
from IPython.display import display, clear_output

# 1. Create the UI Elements
header = widgets.HTML("<h2> Digital Lending Graph Explorer</h2>")
question_input = widgets.Text(
    placeholder='Ask a question (e.g., "What is the cooling-off period?")',
    layout=widgets.Layout(width='70%')
)
ask_button = widgets.Button(
    description='Ask Graph',
    button_style='primary',
    layout=widgets.Layout(width='20%')
)
output_area = widgets.Output(layout={'border': '1px solid #444', 'padding': '10px', 'margin_top': '10px'})

# 2. Define the button click logic
def on_ask_clicked(b):
    with output_area:
        clear_output()
        query = question_input.value
        if query.strip() == "":
            print("Please enter a question.")
            return

        print(f"Searching the graph for: '{query}'...")
        try:
            answer = query_graph_v2(query)
            print(f"\nAnswer:\n{answer}")
        except Exception as e:
            print(f"❌ Error: {e}")

ask_button.on_click(on_ask_clicked)

# 3. Display the Interface
display(header, widgets.HBox([question_input, ask_button]), output_area)
```

```
HTML(value='<h2> Digital Lending Graph Explorer</h2>')
HBox(children=(Text(value='', layout=Layout(width='70%'), placeholder='Ask a question (e.g., "What is the cool…
Output(layout=Layout(border_bottom='1px solid #444', border_left='1px solid #444', border_right='1px solid #44…
```

```
# Save the ultimate version of your compliance engine
nx.write_graphml(G, "Digital_Lending_Final_Logic_Engine.graphml")

# Save a final HTML map so you can show the cluster connections
net.from_nx(G)
net.save_graph("Compliance_Map_Final.html")

print("🏆 Success! Your GraphML 'Logic Engine' and Interactive Map are saved.")
```

```
🏆 Success! Your GraphML 'Logic Engine' and Interactive Map are saved.
```

```
import json
from networkx.readwrite import json_graph

# 1. Export as JSON (Great for Web Apps like D3.js or React)
graph_json = json_graph.node_link_data(G)
with open('rbi_compliance_graph.json', 'w') as f:
    json.dump(graph_json, f, indent=4)

# 2. Export as GraphML (The 'Gold Standard' for Graph Databases)
nx.write_graphml(G, "rbi_compliance_graph.graphml")
```

```
print("💾 Files Saved! Check the 'Output' section in your Kaggle sidebar to download them.")
```

```
💾 Files Saved! Check the 'Output' section in your Kaggle sidebar to download them.
```